

Communication-Matched Warp Collectives for ML-KEM and ML-DSA on a RISC-V SIMT GPU

Anonymous Authors

Abstract—ML-KEM and ML-DSA share Keccak and number-theoretic transforms (NTTs), yet their useful parallelism has incompatible communication: Keccak repeatedly reduces and permutes a 25-word state, while an NTT exchanges coefficient pairs at power-of-two distances. We present communication-matched warp collectives that keep both states in ordinary lane registers of an RV32 RISC-V GPU. The instructions are stateless, use standard source vectors and single-destination writeback, and implement fixed Keccak neighborhoods or paired NTT butterflies. A serialized modular unit shares its routing and multiplier bank across the 16-bit ML-KEM and 32-bit ML-DSA datapaths. A 200-MHz V80 evaluation covers all six standardized parameter sets against a matched, optimized cooperative software mapping. Stage, shared NTT, and arithmetic reuse accelerate complete single-request ML-KEM by $2.314\text{--}2.365\times$ and ML-DSA by $1.910\text{--}2.150\times$; eight-request batches gain $1.914\text{--}2.126\times$. A separate five-point bank sweep selects two multipliers: it removes 87.500% of NTT-hierarchy DSPs at a 2.963% NTTBF test-cycle and at most 0.143% request penalty in middle-set Pointer controls. The results show that GPU cryptographic hardware must be sized at request scope, not by primitive peak throughput.

Index Terms—post-quantum cryptography, ML-KEM, ML-DSA, Keccak, NTT, SIMT, RISC-V

I. INTRODUCTION

The standardization of ML-KEM and ML-DSA gives post-quantum key establishment and digital signatures a common deployment target [1], [2]. Although the two schemes provide different services, both repeatedly invoke SHA-3/SHAKE and polynomial arithmetic. Efficient implementations must also execute sampling, encoding, and, for signatures, input-dependent rejection loops. Accelerating an isolated transform or permutation is therefore insufficient: the useful outcome is lower latency or higher throughput for a complete cryptographic request, including the surrounding computation.

A programmable GPU offers two levels of parallelism for these workloads. Threads can cooperate within a polynomial or a hash state, while independent warps process different requests. GPU implementations of Kyber and Dilithium already exploit this structure through register reuse, kernel fusion, batching, and cooperative scheduling [3], [4]. ML-Cube further combines GPU execution and machine-learning accelerators, including a 25-thread Keccak mapping [5]. These results motivate a complementary question for an extensible SIMT processor: which remaining work merits specialized hardware once the software already exploits warp parallelism?

Our starting point is the cost that remains after software cooperation. On the 200-MHz V80, instrumented complete requests in our cooperative software baseline attribute 52.22%/41.59% of request cycles to Keccak-f and

7.87%/10.63% to NTT/INTT for ML-KEM-768/ML-DSA-65 (Fig. 1). Both families therefore remain useful specialization targets. This baseline keeps Keccak and NTT state in lane registers, with parallel sampling, encoding, and auxiliary arithmetic enabled identically in every hardware comparison. Our reported gains measure specialization beyond that software mapping.

The obstacle is not a lack of arithmetic parallelism but the cost of making operands meet. Each NTT layer contains 128 independent butterflies. With coefficients distributed across lane registers, some partners lie in the same lane and others in lanes separated by an XOR distance. Keccak-f[1600] instead couples 25 64-bit words through column reductions, fixed permutations, and row-neighbor logic [6]. Generic shuffles express both patterns, but introduce instructions and dependencies at each exchange. Keeping the state in registers removes intermediate memory traffic without removing these communication costs. The two primitives thus favor different internal connections even when they share the same architectural register interface.

Prior hardware addresses parts of this problem at other boundaries. RISC-V software optimization [7] and finite-field instruction extensions [8], [9] reduce arithmetic cost; RISQ-V and Nannipieri et al. add coarser polynomial operations to scalar cores [10], [11]. Configurable lattice processors such as Sapphire and KaLi already reuse resources across protocols [12], [13], while KiD explicitly unifies Kyber/Dilithium NTT multiplication [14]. Full-protocol FPGA accelerators also exploit request batching [15]. Sharing arithmetic is therefore established; the issue here is how shared arithmetic consumes distributed SIMT operands and returns results through an existing GPU pipeline.

The hash interface presents a related choice. Prior work quantifies how state transfers and sponge input/output affect hardware hashing performance [16]. Recent RISC-V extensions retain hash state in private registers [17]; a published patent also describes a Keccak round executed collectively by a warp [18]. Building on these interfaces, we study the constraints of an RV32 SIMT core: handling 64-bit state with 32-bit destinations, preserving per-warp dependency tracking, and selecting the instruction granularity at complete-request scope. A faster permutation can have little effect once sampling, sponge I/O, or polynomial work limits execution.

We address three coupled design constraints. First, collective operations must fit ordinary issue and writeback: an NTT butterfly logically has three inputs and two outputs, while a Keccak step mixes values from many lanes. Second,

ML-KEM’s 16-bit and ML-DSA’s 32-bit coefficients require different reductions without duplicating the entire modular datapath. Third, provisioning all 16 pair products in parallel may waste resources when other instructions or warps hide their latency. Neither an isolated primitive benchmark nor a comparison against scalar C resolves these constraints.

We implement *communication-matched warp collectives* in Vortex [19]. The state remains in ordinary GPRs, and each instruction replaces a fixed communication step together with its arithmetic (Fig. 2). Two lanes jointly execute one NTT butterfly, compute one Montgomery product, and each receive one result. A shared serialized multiplier bank handles both moduli and also serves lane-local products. Keccak exposes either three stage operations or one fused round, with explicit low/high outputs on RV32. Every instruction returns a single architectural destination vector and retains no cryptographic state between instructions. We compare these register interfaces with a memory-coupled permutation engine and size the hardware using complete requests.

The resulting Stage-plus-NTT design, including modular-arithmetic reuse, accelerates complete single-request ML-KEM by $2.314\text{--}2.365\times$ and ML-DSA by $1.910\text{--}2.150\times$ over matched cooperative software on a 200-MHz Alveo V80. All six standardized parameter sets are covered. NTT specialization provides additional request-level gains after Keccak acceleration. A separate bank sweep finds that two multipliers replace 16 with at most 0.143% request-cycle increase in its measured workloads. Our contributions are:

- A stateless SIMT instruction interface for Keccak neighborhoods and XOR-paired NTT butterflies, with ordinary operand tracking and one destination per lane; controlled Stage, Round, and Pointer comparisons establish the importance of the execution boundary.
- A shared K/D modular unit that combines pair routing, modulus-specific reduction, and a serialized multiplier bank. It supports forward/inverse NTT and reuses the same arithmetic outside transforms without private polynomial storage.
- A complete-library, six-parameter FPGA evaluation with matched software baselines and separate Keccak, NTT, and arithmetic ablations. Concurrency and five-point multiplier sweeps connect primitive performance to request latency, batch throughput, and routed hardware cost.

II. BACKGROUND AND MOTIVATION

A. PQC Workloads and Common Primitives

ML-KEM establishes keys; ML-DSA provides digital signatures [1], [2]. Both use NTT/INTT for polynomial products and Keccak-f[1600] through SHA-3/SHAKE for hashing and sampling [6]. Their polynomials have 256 coefficients, with $q_K = 3329$ and $q_D = 8380417$; our library mappings use 16-bit KEM and 32-bit DSA coefficients. Keccak updates a 25×64 -bit state on RV32. Requests also include encoding and signing retries.

Figure 1 profiles correct cooperative software A on V80 at 200 MHz with one active warp. Complete requests include

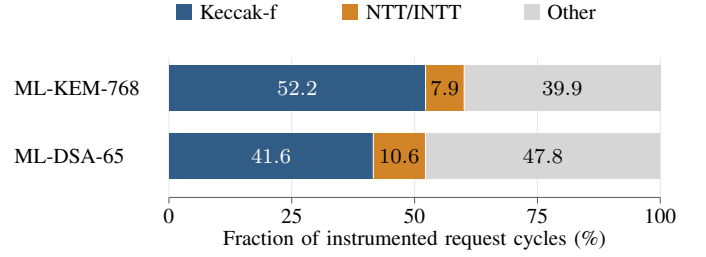


Fig. 1. Instrumented complete-request cycle composition of cooperative software A on the 200-MHz V80.

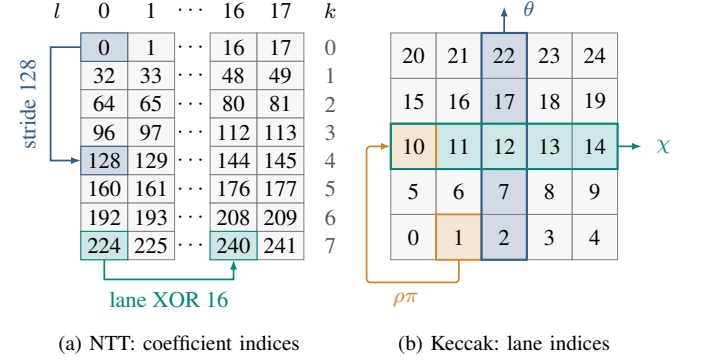


Fig. 2. Register placement determines the communication. (a) Selected NTT lanes: indices 0 and 128 pair within lane 0; 224 and 240 pair across lanes 0 and 16. (b) Keccak uses column parity (θ), a fixed permutation ($\rho\pi$, one edge shown), and row neighbors (χ).

KeyGen, Encaps, and Decaps for KEM, and KeyGen, Sign, and Verify for DSA. We use one KEM KAT input and 32 DSA inputs, with one warmup and five measurements per version. Each operation is timed separately within its complete request.

Disjoint Keccak-f and NTT/INTT intervals are summed across the measured requests and divided by their instrumented total cycles. Keccak timing covers KEM’s in-register permutation and DSA’s permutation wrapper; the latter includes dispatch and state transfer. Sponge absorption and extraction remain in Other. Relative to matched controls without these additional phase probes, profiling adds 4.58% for KEM and 0.90–1.15% across DSA inputs. The plot retains this overhead and identifies major costs rather than an exact uninstrumented breakdown.

B. Warp-Cooperative Execution and Communication

Vortex lanes share an instruction stream over private GPRs [19]. We assign one primitive to a 32-lane warp and independent requests to resident warps. Figure 2 shows the resulting communication. Lane l holds eight coefficients $a[l + 32k]$, $0 \leq k < 8$. The first three forward NTT layers pair registers locally; later layers pair lanes $l \text{ xor } d$, with $d = 16, 8, 4, 2$ for ML-KEM and additionally $d = 1$ for ML-DSA. Inverse traversal reverses this progression.

Keccak places $A[x, y]$ in lane $x+5y$: 25 lanes hold state, and seven are padding. Each word occupies two RV32 GPRs. This layout exposes column reductions, fixed rotation/permutation, and row-neighbor accesses. Other polynomial accelerators use memory rows or private storage [20]–[23].

Software baseline A uses these layouts and the same surrounding warp optimizations as the hardware modes. Register residency removes intermediate state traffic; fixed exchanges still require dependent shuffle and arithmetic instructions. These repeated sequences are our specialization target.

C. Challenges for a Register-Collective ISE

An NTT butterfly consumes two coefficients and a twiddle, producing two coefficients. The target R-type interface names two source registers and one destination. Reconciling these operands requires considering their lane distribution and ordinary writeback. Shared KEM/DSA arithmetic must also preserve different coefficient widths and native reduction conventions.

Keccak’s rotations cross RV32 halves: θ , $\rho\pi$, and a fused round must derive both outputs from the same old (L, H) state. Overwriting one half early can corrupt the other; $\chi\iota$ instead operates on each selected half. Cross-lane operations also require every referenced producer to participate. Section III-A resolves these operand, dependency, and participation constraints.

III. DESIGN AND IMPLEMENTATION

We expose the register layouts in Section II through a collective ISA, then implement specialized Keccak and NTT datapaths behind the core’s ordinary operand and writeback interfaces. ML-KEM and ML-DSA share the modular datapath; Keccak uses a separate fixed-routing unit.

A. Collective ISA

On W32, each collective reads two source-register vectors and writes one ordinary GPR per active lane. State persists in these GPRs: the instruction neither accesses memory nor retains a private cryptographic context. Keccak requires all 32 lanes active, with state in lanes 0–24 and zero results in lanes 25–31. NTT butterflies require a pair-closed mask: either both XOR partners participate or neither does. Lane-local NTTMUL requires no partner.

Table I lists the implemented families. Stage is the primary Keccak design; Round is a fusion comparator. Theta, rho/pi, and Round read both old RV32 halves (L, H) ; chi/iota reads its selected half and a warp-uniform round $r \in [0, 23]$. Only lane zero receives the selected round-constant half. Round encodes r as an immediate. Software preserves the first result in an ordinary temporary until the second instruction has captured the same old pair. Round likewise recomputes from both old halves for each output, without hidden half-round state.

For NTTBF, choose pair-low lane p with bit s clear and partner $h = p \text{ xor } 2^s$. The operation uses $a = a_p$, $b = a_h$, and $\zeta = \zeta_p$; the pair-high ζ_h is unused. The forward CT form computes

$$t = \text{Mont}_Q(b\zeta), \quad (r_p^{\text{CT}}, r_h^{\text{CT}}) = (a + t, a - t). \quad (1)$$

TABLE I
RV32 ISA WITH PER-LANE OPERANDS AND RESULTS. SLASHES DENOTE SEPARATE FORMS: L/H SELECT HALF h ; K/D SELECT MODULUS Q .
NTTBF USES XOR DISTANCE 2^s .

Instruction	$rs_1[l]$	$rs_2[l]$	$rd[l]$
KTHETA.L/H	$A_{L,l}$	$A_{H,l}$	$\theta(A)_h[l]$
KRHOP.L/H	$A_{L,l}$	$A_{H,l}$	$\rho\pi(A)_h[l]$
KCHII.L/H	$A_{h,l}$	r	$\chi\iota_r(A_h)[l]$
KROUND.L/H r	$A_{L,l}$	$A_{H,l}$	$\text{Round}_r(A)_h[l]$
NTTMUL.K/D	a_l	b_l	$\text{Mont}_Q(a_l b_l)$
NTTBF.CT.K/D	a_l	ζ_l	$r_l^{\text{CT}}, (1)$
NTTBF.GS.K/D	a_l	ζ_l	$r_l^{\text{GS}}, (2)$

Thus two source vectors express three butterfly inputs, while one destination vector carries both outputs. Lane-local NTTMUL multiplies its two sources independently in each active lane. The inverse pair $(r_p^{\text{GS}}, r_h^{\text{GS}}) = \text{GS}_Q(a, b, \zeta)$ preserves the libraries’ different conventions:

$$\begin{aligned} \text{GS}_K(a, b, \zeta) &= (\text{Barrett}_K(a + b), \text{Mont}_K((b - a)\zeta)), \\ \text{GS}_D(a, b, \zeta) &= (a + b, \text{Mont}_D((a - b)\zeta)). \end{aligned} \quad (2)$$

K uses signed low-16-bit operands and arithmetic, then sign-extends its results; D uses signed 32-bit arithmetic. Montgomery radices are 2^{16} and 2^{32} , respectively. Outputs preserve the libraries’ native residue representations rather than forcing canonical values in $[0, q)$.

Stage and NTT use custom-0 (opcode $0 \times 0B$); function fields select operation, half, modulus, direction, and $s = \log_2 d \in [0, 4]$ as applicable. Round uses custom-2 ($0 \times 5B$), with r in funct7. Intrinsics emit R-type forms through inline `.insn`, exposing ordinary register dependencies to the compiler.

B. Hardware Implementation

Figure 3 separates core integration, shared K/D arithmetic, and Keccak’s fixed connections. The implementation consumes all lane operands together and requires 32-wide SIMD and ALU datapaths. It does not assume physical reuse of the general shuffle network.

1) *Keccak Datapath*: Figure 3c implements fixed connections among the 25 state lanes. Theta first XOR-reduces each column to C_x , then injects $C_{x-1} \oplus \text{rol}_{64}(C_{x+1}, 1)$ into that column. Rho/pi uses a fixed gather and the source word’s rotation constant. Chi combines neighboring words within a row; iota injects the selected constant into word zero.

Stage has two registered steps. The first captures column parity and the selected input half for theta, the rotated/gathered result for rho/pi, or row logic and the constant for chi/iota. The second completes theta or constant injection; rho/pi passes through. These are alternative paths, not a serial three-operation pipeline. Round instead registers parity, theta, rho/pi, and chi/iota across four steps. Without stalls, both accept one instruction per cycle; unit latencies are two and four cycles, respectively, plus one shared return cycle. Stage requires six RV32 issues per round and Round two. The separate Pointer

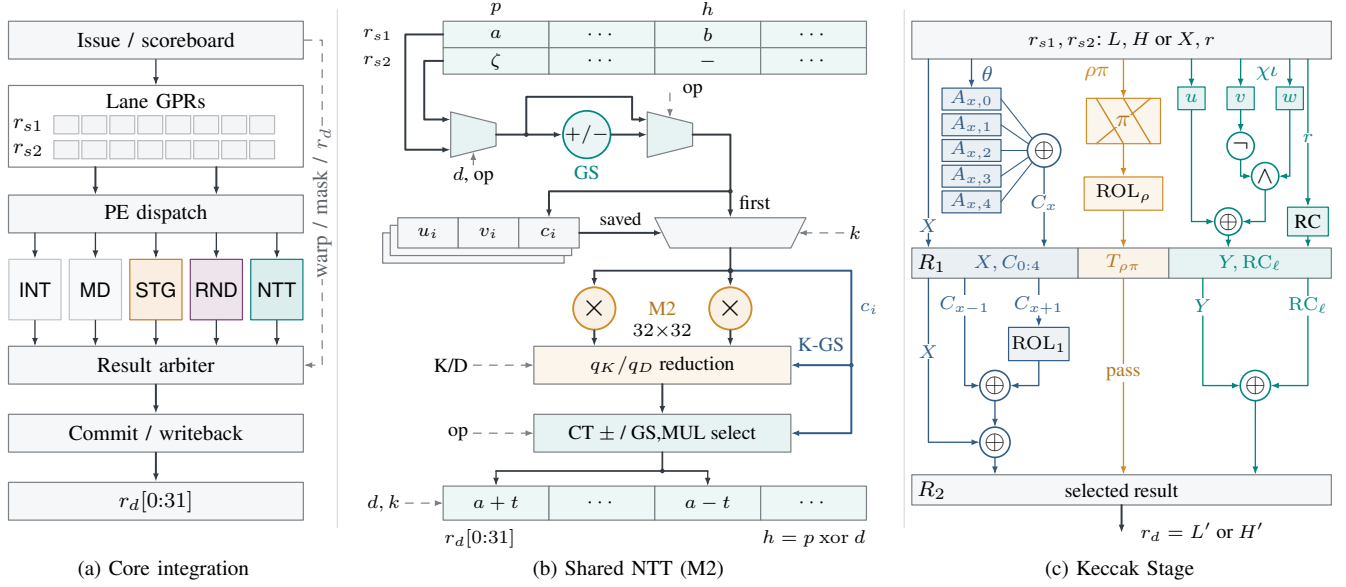


Fig. 3. Core integration and collective datapaths. (a) Dedicated PEs share operand dispatch and result arbitration with INT and multiply/divide. (b) Beat zero bypasses capture; later (u_i, v_i, c_i) groups are replayed. M2 multiplies $u_i v_i$; registered c_i (a or the GS sum) feeds K-GS reduction and finishing. Op selects CT/GS/MUL. CT slots use $t = \text{Mont}_Q(b\zeta)$; the vector returns after the last beat. (c) Three alternative Stage paths cross R_1/R_2 . $A_{x,i} = A[x, i]$ combines L/H ; X is the retained half; u, v, w are row-neighbor halves. Round index r selects RC_ℓ , zero except at lane zero.

comparator loads and stores a 200-byte state through the LSU rather than using the register-collective interface.

2) *Shared K/D Modular Datapath*: Figure 3b separates pair preparation, variable multiplication, modular reduction, and lane-result assembly. K mode sign-extends its inputs and uses Montgomery reduction for $q_K = 3329$; K-GS additionally multiplies each pair sum by 20159 for Barrett reduction. D uses the full signed product and its own Montgomery constants for $q_D = 8380417$. Mode selection controls preparation and reduction, while capture, multipliers, beat scheduling, coefficient bypass, and assembly remain shared. NTTMUL selects lane-local inputs for the same datapath. Explicit twiddles require no internal twiddle ROM or transform-layer state.

A bank of M signed 32×32 multipliers serializes logical products without changing the instruction or binary; M counts general product multipliers, not FPGA DSP blocks. CT and D-GS require $B = 16/M$ beats. K-GS requires $B = 32/M$ for twiddle and Barrett products; 32-lane NTTMUL also uses $32/M$. Inactive pairs do not compress these slots. Figure 4a shows M2 assigning 16 CT products to eight beats. Products traverse factor and reduction registers; retained coefficients bypass multiplication into butterfly finishing.

3) *Execution Control and Core Integration*: Figure 3a places Stage, Round, and NTT beside the integer and multiply/divide PEs. Decode names the ordinary source and destination registers for scoreboard tracking; ALU dispatch selects the PE, and a round-robin result arbiter feeds the existing commit path. No additional register-file port or private architectural register bank is required.

Figure 4b shows the resulting instruction overlap. A serialized NTT instruction feeds the first product group and captures the remaining groups, retained coefficients, and warp metadata

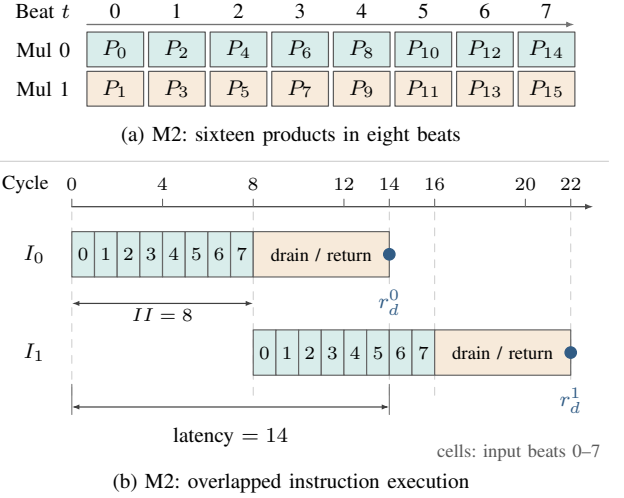


Fig. 4. M2 serialization. (a) At beat t , the multipliers accept pair products P_{2t} and P_{2t+1} , so CT/D-GS use eight beats. (b) I_1 begins as I_0 drains: the initiation interval is 8 cycles and the acceptance-to-commit latency is 14 cycles. K-GS/NTTMUL use 16 beats.

on its first handshake. Saved operands feed subsequent beats; a new instruction can enter after those beats launch while its predecessor drains. Warp, destination, mask, and beat indices follow the products. Indexed assembly collects partial results and releases one destination vector after the last beat. Backpressure freezes the serializer, pipeline, and assembly together. Without stalls, initiation interval is B and latency including the shared return is $B + 6$ cycles: M2 CT takes eight beats and 14 cycles. The destination remains pending in the scoreboard until the vector returns; intermediate beats

never update architectural registers. Other ready warps can issue to available PEs during this interval. Retained operands and partial results are in-flight state, not persistent accelerator-private context.

C. Software Integration

Native hooks preserve the mlkem-native/mldsa-native APIs and byte-exact outputs across all six parameter sets [24], [25]. Lane zero broadcasts arguments, enters the cooperative region, and resumes scalar control afterward. Each resident warp handles one request at a time; the leader controls allocation and signing rejection. One-shot SHAKE retains state in registers across absorb, permutation, and squeeze; incremental contexts are stored between calls. NTT loads eight coefficients per lane and completes the transform without intermediate memory, using lane-local multiplication or XOR-paired butterflies as required. Kernels enforce participation and old-half dependencies.

Transform-external reuse preserves reduction boundaries. DSA pointwise products use NTTMUL.D; matrix-vector products first accumulate $L \in \{4, 5, 7\}$ raw products in 64-bit software, then perform one Montgomery reduction per coefficient. KEM basemul retains software products and accumulation, optionally reusing NTTMUL.K for Montgomery correction. Thus reuse does not replace every raw product with an independently reduced product.

RTL assertions and unit/processor tests cover participation, both moduli, XOR distances, dependencies, metadata, and backpressure. Library outputs match portable C byte for byte; SimX models the same beats and stalls, with matched binaries checked through XRT integration.

IV. EVALUATION

A. Experimental Methodology

We evaluate the extensions on an Alveo V80 FPGA with one RV32IM Vortex core operating at 200 MHz. The core supports eight resident 32-lane warps, with 16 KiB each of instruction cache, data cache, and local memory. The data cache is write-through, L2/L3 are disabled, and the shared NTT bank contains two multipliers. FPGA implementations use Vivado 2025.1; the independent core implementations use OPT3. We verify the board clock before and after each measurement.

The workloads cover ML-KEM-512/768/1024 and ML-DSA-44/65/87. Each request executes KeyGen, Encaps, and Decaps for KEM, or KeyGen, Sign, and Verify for DSA. Baseline A uses the cooperative software mapping described in Section III. B adds Stage, C adds shared NTT, D combines both, and E extends D with NTTMUL outside transforms. All modes retain the same surrounding software optimizations and zeroization policy and execute matched inputs on the same board image. Opcode audits confirm that A contains no PQC instructions and A/B contain no NTT instructions. Portable C provides correctness references; PQRV's scalar Keccak [7] is used only in separate controls.

TABLE II
COMPLETE-REQUEST SPEEDUP OVER A AT 200 MHz ($N = 1$).
 $S_X = C_A/C_X$, WHERE C_X IS THE MEDIAN CYCLE COUNT OF MODE X ;
BOLD MARKS FINAL MODE E.

Parameter set	S_B	S_C	S_D	S_E
KEM-512	2.016	1.073	2.324	2.341
KEM-768	2.048	1.061	2.316	2.314
KEM-1024	2.124	1.050	2.353	2.365
DSA-44	1.495	1.095	1.715	1.966
DSA-65	1.466	1.097	1.684	1.910
DSA-87	1.744	1.067	1.950	2.150

TABLE III
CUMULATIVE BATCH SPEEDUP OVER A AT 200 MHz FOR $N = 8$
CONCURRENT REQUESTS. S_X FOLLOWS TABLE II; BOLD MARKS FINAL
MODE E.

Parameter set	S_B	S_C	S_D	S_E
KEM-512	1.848	1.063	2.077	2.126
KEM-768	1.827	1.050	2.007	2.004
KEM-1024	1.835	1.043	1.986	2.013
DSA-44	1.514	1.123	1.808	1.966
DSA-65	1.541	1.120	1.839	1.914
DSA-87	1.687	1.088	1.946	2.023

We measure on-device cycles from the earliest request start to the latest completion, excluding host transfers and setup outside this interval. Batch measurements retain any gaps between execution waves. Each configuration is run once for warmup and five times for measurement; reported speedups compare median cycle counts against A for the same batch size. These runs omit the additional phase probes used in Fig. 1. Separate fusion and bank studies retain their archived write-back configurations and timing windows, specified with their results.

All 156 board configurations (780 measured runs) pass byte-exact checks against portable C. RTL tests cover both moduli, transform directions, lane pairing, masks, and backpressure. Across the 60 single/eight-request A–E pairs, SimX and XRT RTL integration retire identical instruction counts and differ by at most 4.422% in device cycles. The board runs also match the XRT instruction counts.

B. Complete-Request Acceleration

Tables II and III report every cumulative mode for all six sets. E accelerates one-request KEM/DSA by 2.314–2.365 $\times$ /1.910–2.150 $\times$ over A; at eight requests the ranges are 2.004–2.126 $\times$ /1.914–2.023 $\times$. E completes a single request in 1.182–2.951 Mcycles for KEM and 10.253–21.436 Mcycles for DSA.

The same measurements isolate each hardware contribution. Across all sets and one/eight requests, Stage gives 1.827–2.124 $\times$ for KEM and 1.466–1.744 $\times$ for DSA (A/B). With Stage fixed, shared NTT adds 1.082–1.153 $\times$ /1.118–1.194 $\times$ (B/D), so NTT retains request-level value. Modular reuse outside transforms adds 1.040–1.146 $\times$ for DSA without new RTL, but only 0.998–1.024 $\times$ for KEM (D/E); KEM-768 is within 0.2% of D. Headlines use E. Within one case,

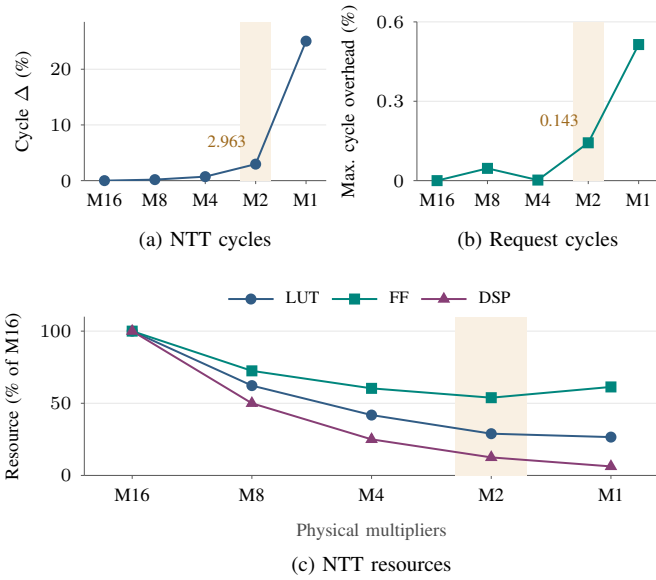


Fig. 5. NTT multiplier-bank tradeoffs. (a) Historical W4T32 NTTBF.K test cycles. (b) Maximum W8T32 XRT request-cycle overhead across KEM-768/DSA-65 and $N = 1/8$, with Pointer Keccak fixed. (c) Independent 200-MHz NTT resources. M16 is the bank reference; gold marks M2.

$(A/B)(B/D)(D/E) = A/E$; separate cohorts are not combined.

C. Granularity and Arithmetic Capacity

We compare Round with equally unrolled Stage using XRT permutation intervals. Fusion improves dependent permutation throughput by $3.168\times/3.291\times$ at one/eight warps, but separate SimX complete-KEM gains are only $1.028\times/1.033\times$; the latter has no paired XRT run. An instrumented XRT control attributes only 4.964% of a request to Stage permutation. A separate fixed-mapping control finds Round 1.130 $\times$ faster than Pointer at one request, whereas Pointer is 1.374 $\times$ faster at eight. Thus fusion and offload must be evaluated at request scope and target concurrency.

Figures 5(a,b) distinguish test-program cycles from complete requests. The former uses historical RV32IMF W4T32 processor RTL, including loads, stores, and loops. The latter fixes Pointer Keccak in RV32IM W8T32 write-back XRT runs for KEM-768/DSA-65 at one/eight requests. Each timing experiment preserves its binary and inputs across banks.

M16 minimizes NTTBF test cycles. M8 and M4 increase them by 0.181% and 0.715%; M2 increases them by 2.963%, with at most 0.143% request-cycle overhead. The independent routed study in Fig. 5(c) finds that M2 removes 71.074% of NTT LUTs, 46.131% of FFs, and 87.500% of DSPs. M1 saves only five more core DSPs and increases NTT FFs, while test cycles rise by 25.043% over M16. M2 retains most resource savings with little measured request penalty.

All five NTTBF tests retire 59,534 instructions, pass 1,044 vectors, and stay within 0.959% SimX/RTL difference. Request sensitivity is much smaller than test-program sensitivity;

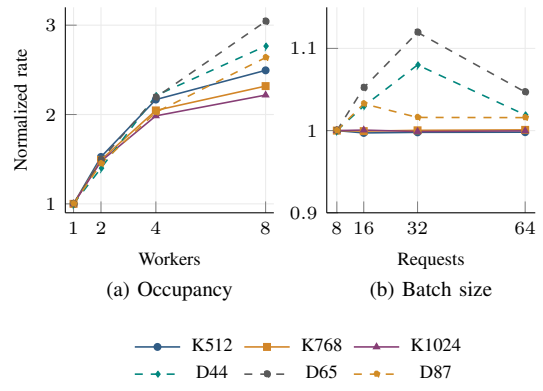


Fig. 6. Normalized 200-MHz throughput for E across six sets: (a) eight inputs versus workers, normalized at $W = 1$; (b) batch size at $W = 8$, normalized at $N = 8$.

TABLE IV
INDEPENDENT 200-MHZ POST-ROUTE RV32IM CORES; M16 VARIES
KECCAK AND M2 IS MEASURED WITH AND WITHOUT STAGE.

Variant	LUT	FF	DSP	WNS (ns)
M16 base	233,759	136,451	176	+0.078
M16 + Stage	237,073	138,503	176	+0.116
M16 + Round	236,625	142,541	176	+0.052
M16 + Pointer	243,399	139,407	176	+0.080
M2 base	223,964	131,306	106	+0.048
M2 + Stage	226,852	133,368	106	+0.145

matched instruction counts exclude a software change but do not isolate latency hiding.

D. Concurrency and Batch Size

Figure 6 separates occupancy from batch size. Throughput for E is $R = Nf/C_E$ requests/s at $f = 200$ MHz, using its median batch makespan; the figure reports R/R_{ref} . The reference is $W = 1$ at fixed $N = 8$ in (a), and $N = 8$ at fixed $W = 8$ in (b). On eight fixed inputs, increasing workers from one to eight raises E throughput by 2.218–3.042 $\times$. With eight workers fixed, extending batches from eight to 64 changes throughput to only 0.998–1.047 $\times$ its eight-request value. Intermediate DSA batches peak at 1.119 $\times$, so throughput is not monotonic. Batches use synchronized waves. DSA signing retries vary by input; extending a batch also extends its input prefix, so its curve includes both amortization and workload variation.

E. Routed Cost and Design Scope

All cores in Table IV close without routing errors. M2 reduces whole-core LUTs/DSPs by 4.190%/39.773% over M16; Stage adds 1.289% LUTs to M2. At M16, Round uses fewer LUTs but more FFs than Stage, so neither dominates in resource cost. The board AFU also includes optional engines: 243,073 LUTs, 181,881 FFs, 106 DSPs, and 0.001 ns setup slack. Board ratios compare software paths within this image.

Six-parameter results validate Stage+M2, without repeating the bank sweep or establishing joint optimality. Other trans-

form intensity or concurrency may favor another design point. Energy and host-to-host latency are unmeasured.

V. CONCLUSION

We presented stateless warp collectives that expose Keccak neighborhoods and XOR-paired NTT butterflies through ordinary lane registers. A shared K/D modular datapath supports both coefficient widths and reuses its multiplier bank beyond transforms, while retaining standard operand tracking and single-destination writeback. On a 200-MHz V80, Stage+M2 with modular reuse accelerates complete requests across all six parameter sets by $1.910\text{--}2.365\times$ for one request and $1.914\text{--}2.126\times$ for eight-request batches relative to matched cooperative software.

Independent routed comparisons show that M2 removes 87.500% of NTT-hierarchy DSPs relative to M16. A separate bank sweep with Pointer Keccak on KEM-768/DSA-65 measures at most 0.143% request-cycle overhead for M2. Together with the granularity study, these results support choosing collective operations and arithmetic capacity against complete-request cost and target concurrency. The design integrates specialized communication and modular arithmetic into a programmable SIMT core while preserving its register interface and surrounding software mapping.

REFERENCES

- [1] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” NIST, Tech. Rep. FIPS 203, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [2] —, “Module-lattice-based digital signature standard,” NIST, Tech. Rep. FIPS 204, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [3] X. Ji, J. Dong, T. Deng, P. Zhang, J. Hua, and F. Xiao, “HI-Kyber: A novel high-performance implementation scheme of Kyber based on GPU,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 6, pp. 877–891, 2024.
- [4] S. Shen, H. Yang, W. Dai, H. Zhang, Z. Liu, and Y. Zhao, “High-throughput GPU implementation of Dilithium post-quantum digital signature,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 11, pp. 1964–1976, 2024.
- [5] T. Zhou, F. Zheng, Z. Xie, W. Tang, G. Fan, Y. Ning, Y. Bian, J. Lin, and J. Jing, “ML-Cube: Accelerating module-lattice-based cryptography using machine learning accelerators with a memory-less design,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2025, pp. 1829–1843.
- [6] National Institute of Standards and Technology, “SHA-3 standard: Permutation-based hash and extendable-output functions,” NIST, Tech. Rep. FIPS 202, 2015. [Online]. Available: <https://csrc.nist.gov/pubs/fips/202/final>
- [7] J. Zhang, Y. Yan, J. Huang, and Ç. K. Koç, “Optimized software implementation of Keccak, Kyber, and Dilithium on RV{32,64}IM{B}{V},” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 1, pp. 632–655, Dec. 2024.
- [8] E. Alkim, H. Evkan, N. Lahr, R. Niederhagen, and R. Petri, “ISA extensions for finite field arithmetic: Accelerating Kyber and NewHope on RISC-V,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 3, pp. 219–242, 2020.
- [9] R. Bevin, A. Khalid, M. Imran, and M. O’Neill, “Accelerating CRYSTALS-Kyber and Dilithium via a single Montgomery reduction ISE on RISC-V,” in *Proc. IEEE 38th Int. System-on-Chip Conf. (SOCC)*, 2025, pp. 1–6.
- [10] T. Fritzmann, G. Sigl, and J. Sepúlveda, “RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 4, pp. 239–280, 2020.
- [11] P. Nannipieri, S. D. Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci, “A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms,” *IEEE Access*, vol. 9, pp. 150 798–150 808, 2021.
- [12] U. Banerjee, T. S. Ukyab, and A. P. Chandrakasan, “Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2019, no. 4, pp. 17–61, 2019.
- [13] A. Aikata, A. C. Mert, M. Imran, S. Pagliarini, and S. S. Roy, “KaLi: A crystal for post-quantum security using Kyber and Dilithium,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 2, pp. 747–758, 2023.
- [14] S. Mandal and D. B. Roy, “KiD: A hardware design framework targeting unified NTT multiplication for CRYSTALS-Kyber and CRYSTALS-Dilithium on FPGA,” in *Proc. 37th Int. Conf. VLSI Design and 23rd Int. Conf. Embedded Systems (VLSID)*, 2024, pp. 455–460.
- [15] X. Carril, C. Kardaris, J. Ribes-González, O. Farràs, C. Hernandez, V. Kostalabros, J. U. González-Jiménez, and M. Moretó, “Hardware acceleration for high-volume operations of CRYSTALS-Kyber and CRYSTALS-Dilithium,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 17, no. 3, pp. 41:1–41:26, 2024.
- [16] P. Karl, J. Schupp, and G. Sigl, “Performance and communication cost of hardware accelerators for hashing in post-quantum cryptography,” *ACM Transactions on Embedded Computing Systems*, vol. 24, no. 5, pp. 66:1–66:31, 2025.
- [17] A. Dolmeta, V. Piscopo, M. Hutter, M. Martina, and G. Masera, “HORCRUX: A complete PQC RISC-V extension architecture,” *arXiv:2607.13939*, 2026, preprint.
- [18] M. B. Sullivan, S. Ghosh, D. R. Johnson, G. Suh, C. Joseph, and M. Y. Siu, “Warp-collective instructions to accelerate cryptographic primitives,” U.S. patent application publication US 2026/0222190 A1, Jul. 2026, applicant: NVIDIA Corp.; published July 30, 2026, Listing 3. [Online]. Available: <https://patents.justia.com/patent/20260222190>
- [19] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3d-graphics research,” in *Proc. 54th Annual IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2021, pp. 754–766.
- [20] J. Park, S. Lee, and J. Lee, “NTT-PIM: Row-centric architecture and mapping for efficient number-theoretic transform on PIM,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [21] J. Zhang, M. Imani, and E. Sadredini, “BP-NTT: Fast and compact in-SRAM number theoretic transform with bit-parallel modular multiplication,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [22] X. Ren, Z. Chen, Z. Gu, Y. Lu, R. Zhong, W. jie Lu, J. Zhang, Y. Zhang, H. Wu, X. Zheng, H. Liu, T. Chu, C. Hong, C. Wei, D. Niu, and Y. Xie, “CHAM: A customized homomorphic encryption accelerator for fast matrix-vector product,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [23] A. Putra, C. M. Duong, A. Q. Pham, and J.-Y. Kim, “ZK-Flex: A flexible and scalable framework for accelerating zero-knowledge proofs,” in *Proc. Design Automation Conf. (DAC)*, 2026.
- [24] PQ Code Package contributors, “mlkem-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mlkem-native>
- [25] —, “mldsa-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mldsa-native>